

Shared GKE Tenant Onboarding Guide

Introduction

Welcome to the onboarding guide for the **Shared GKE Platform**. This document is intended to help new application teams (Tenants) onboard their services onto DevEx Multi-Tenant Kubernetes Clusters (Shared GKE Clusters) hosted on GCP's Google Kubernetes Engine(GKE).

This guide provides:

- Understanding of which shared cluster a tenant will be part of.
- An overview on DevEx CI/CD pipeline using GitHub Actions.
- Configuration standards and GitHub repo structure.

By the end of this guide, your team should be able to

- Deploy the GKE with minimal manual intervention.
- Manage and maintain your manifests repos in a structured, scalable manner.
- Leverage DevEx tools and automation.

 If at any point you need help with CI/CD setup (or) patterns, reach out to DevEx platform team #devex-compute-customers, DevSecOps team #pcw-devex-devsecops-engagement-team, DevEx CI/CD team #community-cicd channels on Slack.

DevEx mission statement: devex-cicd-doc.cvshealth.com

Manifest Repo

This manifest repository holds all GKE deployment configurations and namespace related configurations for every tenant onboarding to their respective LOB specific shared cluster. Each cluster will be having a manifest repository, where it holds the Kubernetes manifests for the resources (applications, namespace-config, services etc) deployed on that cluster.

You can find your cluster's manifest repo location [here](#).

Note: Ensure you have a branch created for your namespace on your cluster specific Manifest repo, If not, you may need to create a separate branch using [this](#) automation.

Namespace Configuration

Namespace configuration is nothing but a branch name of the manifest repo. Each branch represents a namespace in that cluster.

If you don't see a branch name which matches the namespace where you want to deploy your application, then create a branch with the namespace name.

You should use [this](#) automation to create a namespace branch in your manifest repo.

The screenshot shows the GitHub Actions interface for the 'Create GKE Namespace Configuration' workflow. The workflow is triggered by a 'workflow_dispatch' event. The interface displays a list of 36 workflow runs, all of which are successful. A modal window is open on the right, showing the 'Run workflow' button and various configuration options like 'Branch: main', 'LOB (Line of Business):', 'Data Classification:', 'Environment:', 'Compute Project:', 'API ID:', 'Namespace Sizing:', and 'Namespace Owner emailID:'.

You can refer [this](#) self service guide for GKE namespace config in manifest repo (you still need to follow entire document in order to create actual namespace in respective gke cluster).

Note:

- It is recommend that an app team should have only one namespace config in manifest repo.

edml_restricted_gke-gke_manifests Internal Watch 0

scratch-two-dev 9 Branches 0 Tags Go to file Add file Code

Switch branches/tags

Find or create a branch...

Branches Tags

main default

edp-ss

renovate/configure

scratch-dev

✓ scratch-two-dev

eb-maven-scratch cleanup: removed old ver... a85ce22 · 3 months ago 43 Commits

gha-java-web-maven-scratch deploy manifests id: 14 3 months ago

Update scratch-two-dev.yaml 3 months ago

gha-java-web-maven-scratch cleanup: removed old versi... 3 months ago

Initial commit 4 months ago

Contribute

Application Endpoints Configuration

- Make sure application endpoints are added to Istio config provided in namespace-config manifest.
- You can use [this](#) automation for adding Istio endpoint to your manifest repo.

Actions New workflow

All workflows

Create GKE Manifest Repository

Create GKE Namespace

Istio Application Endpoint - Add

Istio Application Endpoint - Delete

Management

Caches

Attestations

Runners

Usage metrics

Performance metrics

Istio Application Endpoint - Add

add-istio-endpoint.yaml

Filter workflow runs

3 workflow runs

This workflow has a workflow_dispatch event trigger.

Run workflow

Use workflow from

Branch: main

LOD (Line of Business):

edml

Data Classification:

restrict

Environment:

dev

API ID:

Application URL:

Run workflow

Files

scratch-two-dev + Go to file

applications

namespace-configs

scratch-two-dev.yaml

services

README.md

edml_restricted_gke-gke_manifests / namespace-configs / scratch-two-dev.yaml

sudarsanan-rengarajan_cvsh Update scratch-two-dev.yaml

Code Blame 23 lines (19 loc) · 649 Bytes

```

1 #argocd related data
2 argo:
3   project: default
4   manifestRepo: "https://github.com/cvs-health-source-code/edml_restricted_gke-gke_manifests.git"
5   notifyEmailList: "sudarsanan.rengarajan@cvshealth.com"
6
7 #istio related values
8 istioInternalHosts: |
9   - "internal-non-prod.edml-restricted-gke.cvshealth.com"
10  - "edp-ss.dev.cvshealth.com"
11
12 # istioExternalHosts: |
13 #   - "test.com"
14
15 #istio Host details for Istio auth
16 istioHosts: ["internal-non-prod.edml-restricted-gke.cvshealth.com", "edp-ss.dev.cvshealth.com"]
17
18 #namespace resources default and limits
19 namespaceDetails:
20   defaultMemory: "350Mi"
21   defaultCpu: "100m"
22   maxMemory: "1Gi"
23   maxCpu: "2"

```

Note: Ensure to use cvshealth.com domain for all edp shared GKE Clusters , DevEx not allowing to add *.aig.aetna.com domains .

- Raise a DNS request for application endpoints. DNS request can be submitted [here](#).

Note: The IP on the below request is the Ingress IP address of the GKE cluster “edml-restricted-gke” on GCP project “edp-dev-restrict-edml-gke”. You should use the Ingress IP of your respective GKE cluster. Ingress IP can be found [here](#) for your respective cluster.

DNS Request Details

Action	DNS Record Type	DNS Source	DNS Destination
Add	A: Address Record	edp-ss.dev.cvshealth.com	10.112.152.2

Close

ActivityAttachme...Additional ...

User name or ID

Sudarsanan Rengarajan

Organizational Unit

Health Care Business (HCB)

Request Preference

ASAP

Environment

Internal

DNS Request Details

Click to view

Automation DNS Request Details

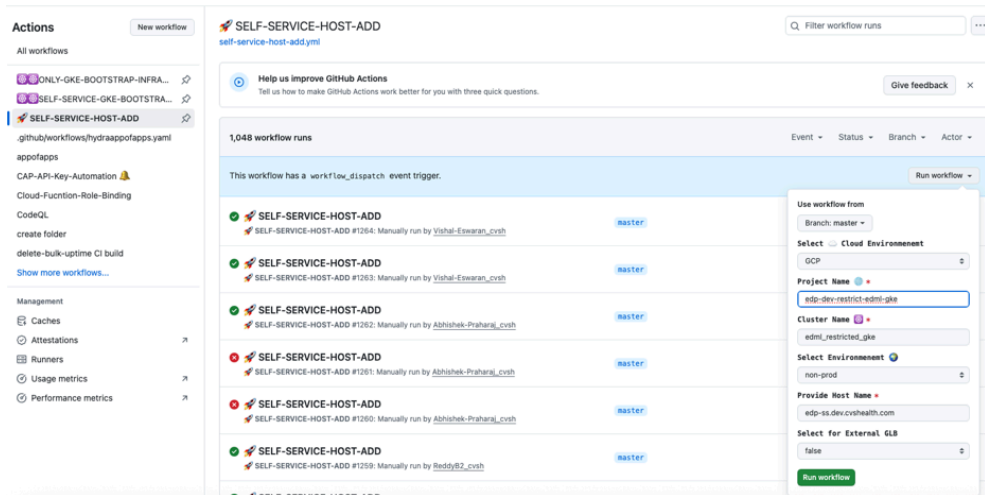
Click to view

Additional Information / Special Instructions

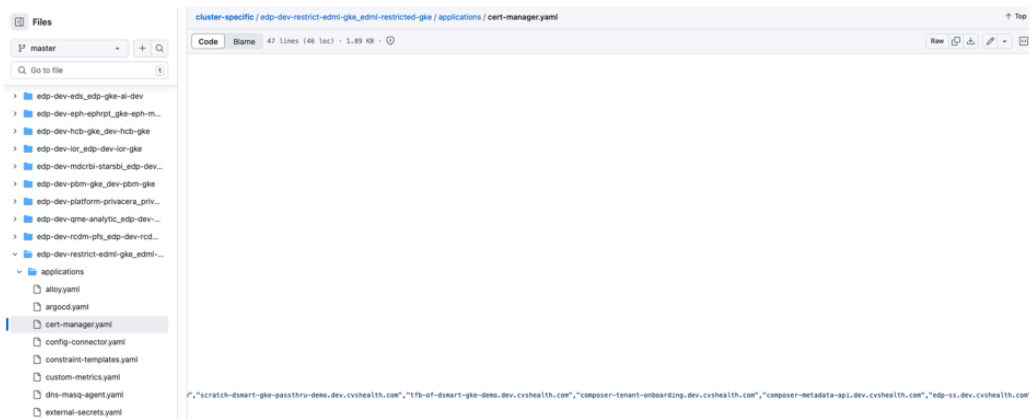
This DNS is for a edp shared services application deployed in shared "edml_restricted_gke" GKE cluster in the namespace "scratch-two-dev", located in GCP project "edp-dev-restrict-edml-gke"

-  Make sure DNS mapping done correctly before proceeding workflow automation .
you can verify the DNS mapping by checking <https://dnsq.dev.cvshealth.com/>

- Run this workflow [automation](#) for cert manager configuration for that new endpoint.
Make sure DNS mapping done correctly before proceeding workflow automation .
you can verify the DNS mapping by checking <https://dnsq.dev.cvshealth.com/>



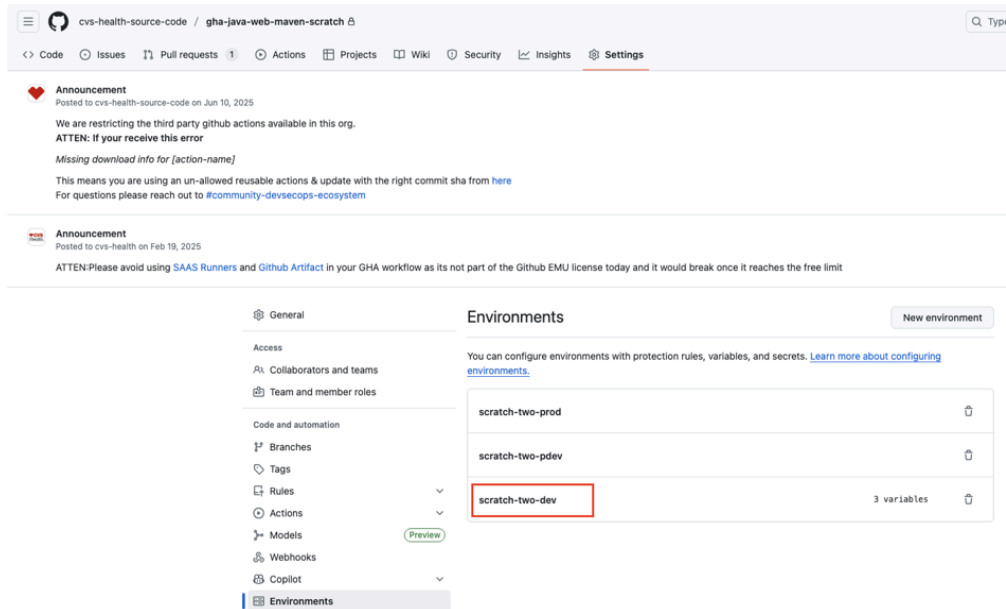
- Ensure that post workflow run, your new domain gets added to the `cert-manager.yaml` in `cluster-specific` repo of your cluster.



One Time Setup for Every Repository

Follow the below steps specified to configure the one time setup for each repository for your team. You need to have **admin** access to your repository to configure the below steps :

1. To create a new environment, go to the repository's **Settings**, select **Environments** from the left menu, click **New environment**, and enter your namespace name (e.g., `scratch-two-dev`).



2. To configure environment variables, select the desired environment (e.g., **scratch-two-dev**), click **Add environment variable**, and define the following variables with cluster specific values:

- **CLUSTER_NAME**
- **CLUSTER_TYPE**
- **MANIFESTS_REPO_NAME**

Ensure the variable name as following the same naming standard and conventions.

Environment variables

Add environment variable

Variables are used for non-sensitive configuration data. They are accessible only by GitHub Actions in the context of this environment by using the [variable context](#).

Name	Value	Last updated		
CLUSTER_NAME	edml-restricted-gke	3 months ago		
CLUSTER_TYPE	GKE	4 months ago		
MANIFESTS_REPO_NAME	edml_restricted_gke-gke_manifests	2 months ago		

3. To configure Repository secret, select “**Secrets and variables**” from the left menu and select the sub-menu “**Actions**”, click “**New repository secret**” and add the secret name in the pattern “**<<CLUSTER_NAME>>_ARGOPASSWORD**” based on the cluster name, and configure it with the admin password of ArgoCD.

Environment secrets

This environment has no secrets.

[Manage environment secrets](#)

Repository secrets [New repository secret](#)

Name	Last updated
EDML_RESTRICTED_GKE_ARGOPASSWORD	4 months ago

Organization secrets

Name	Last updated
APOLLO_KEY	4 months ago
ARGOPASS	2 years ago
ARTIFACTORY_DEVSECOPS_ATTEST_TOKEN	2 months ago
ARTIFACTORY_DEVSECOPS_ATTEST_USER	8 months ago
ARTIFACTORY_JAVA_SETTINGS_XML_TEMPLATE	2 months ago
ARTIFACTORY_PASSWORD	7 months ago
ARTIFACTORY_TOKEN	2 months ago

4. To configure Repository variables, select “**Variables**” tab next to secrets tab, click “**New repository variable**” and add the variables name in the pattern “<<CLUSTER_NAME>>_ARGOURL”, “<<CLUSTER_NAME>>_ARGOUSER” based on the cluster name, and added it with the related ArgoCD configuration values.

Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Environment variables [Manage environment variables](#)

Name	Value	Environment	Last updated
CLUSTER_NAME	edml-restricted-gke	scratch-dev	3 months ago
CLUSTER_TYPE	GKE	scratch-dev	4 months ago
MANIFESTS_REPO_NAME	edml_restricted_gke-gke_manif...	scratch-dev	2 months ago

Repository variables [New repository variable](#)

Name	Value	Last updated
EDML_RESTRICTED_GKE_ARGOURL	internal-argocd-non-prod.edml-restricted...	4 months ago
EDML_RESTRICTED_GKE_ARGOUSER	admin	4 months ago

Organization variables

Name	Value	Last updated
ARTIFACTORY_DEVSECOPS_PASSWORD	non-sensitive-but-dangerous	8 months ago

5. Ensure all the custom properties are added as per devex standards <https://aetnao365.sharepoint.com/sites/Application-Ecosystem-Security-and-Threat-Management/SitePages/Application-Mapping.aspx> Connect your OneDrive account

GitHub Repo Structure & Standards

Source Code Repo

Each application team is responsible for maintaining a dedicated source code repository, which contains the application code base and CI/CD workflows configurations.

Path for holding CI workflows: ***.github/workflows***

CI

Note: make sure to always use “@latest” for all the deployments (node/maven/python) in CI.yaml . this is to ensure you are using always devex latest workflows.

Recommended Folder Layout for CI workflow

```
1 name: Run Docker Pipeline
2 on: [workflow_dispatch]
3 jobs:
4   docker_build:
5     uses: cvs-health-source-
code/gha_workflow_actions/.github/workflows/maven_docker.yaml@latest
6   with:
7     JAVA_VERSION: "20"
8     JAVA_BUILD_COMMAND: "mvn clean package"
9     SAST_TOOL: snyk
10    SCA_TOOL: snyk
11    secrets: inherit
```

DevEx Documentation: [CI Workflow Step-By-Step Process](#)

Sample Projects

Java Tech Stack Example

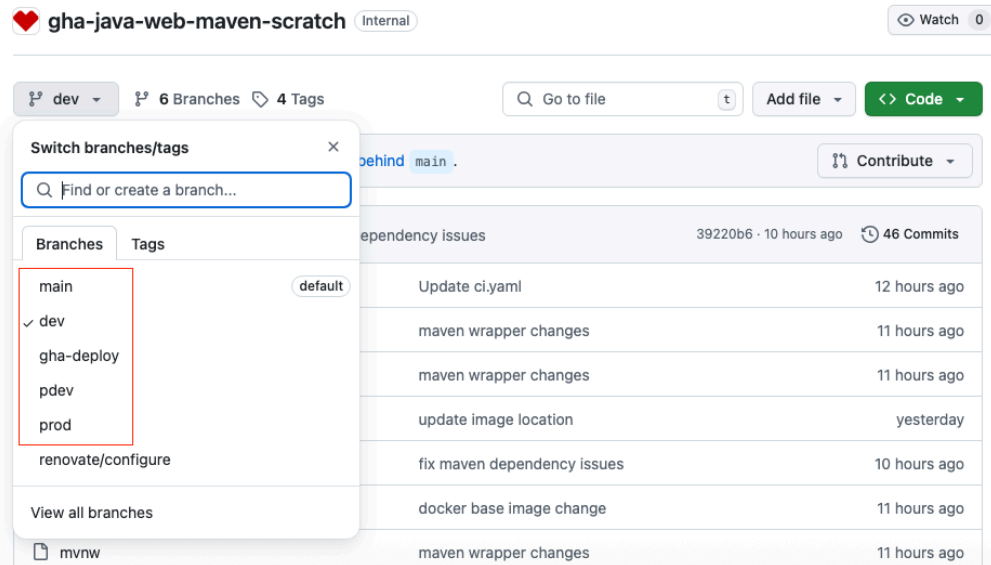
<https://github.com/cvs-health-source-code/gha-java-web-maven-scratch> Connect your Github account

Python Tech Stack Example

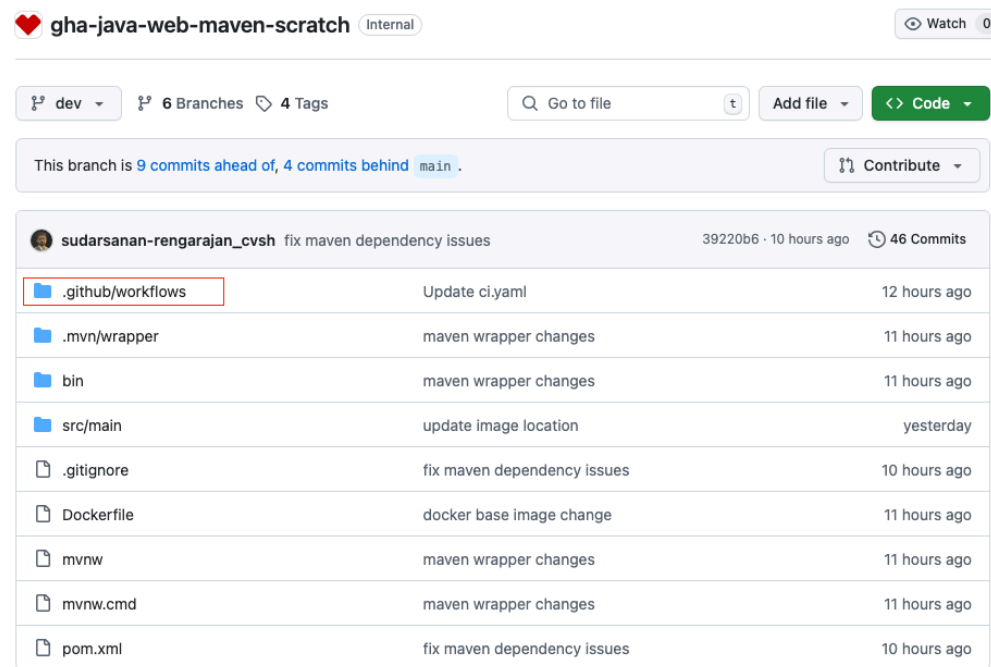
<https://github.com/cvs-health-source-code/scratch-340b-dsmart-demo> Connect your Github account

<https://github.com/cvs-health-source-code/rxomni-drug-search> Connect your Github account

1. Create branches for your environment dev/qa/pdev/pqa/prod (Will explain about gha-deploy branch during the CD process)

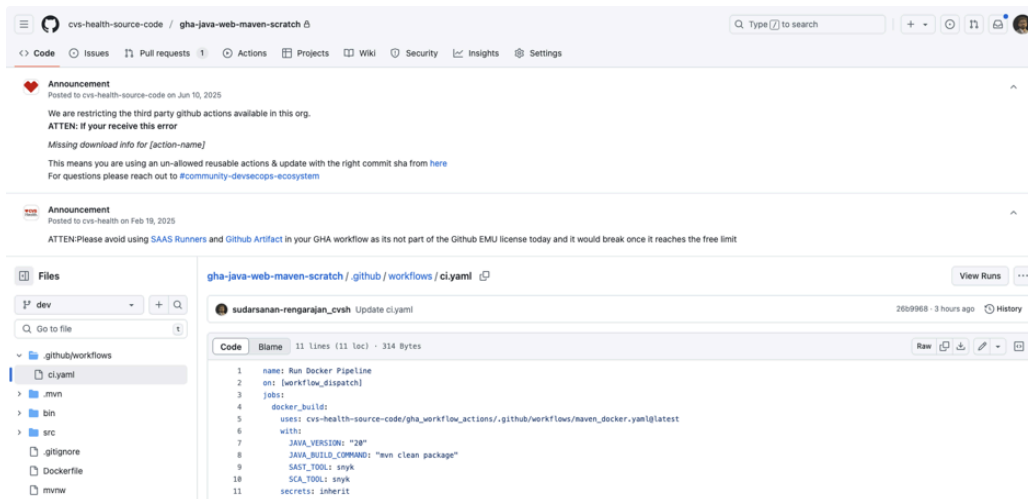


2. Create **.github/workflows** in root directory of the respective env branch.



3. Create a ci.yaml file in **.github/workflows** directory.

Refer this [reusable workflows](#) based on the tech stack you use for your application, in this example we have used maven_docker workflow yaml.

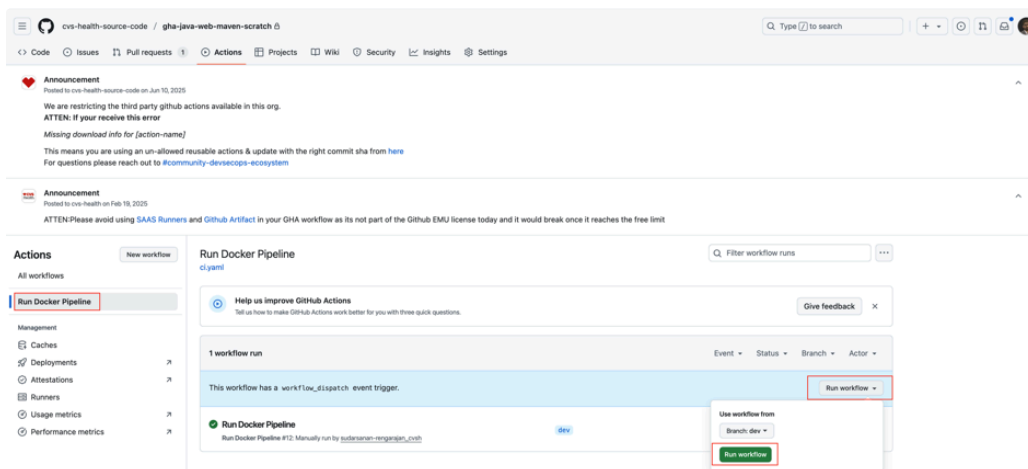


Jfrog

Document for JFrog access: [DevEx Jfrog Artifactory Access and Life Cycle Policy | Jfrog Access groups](#)

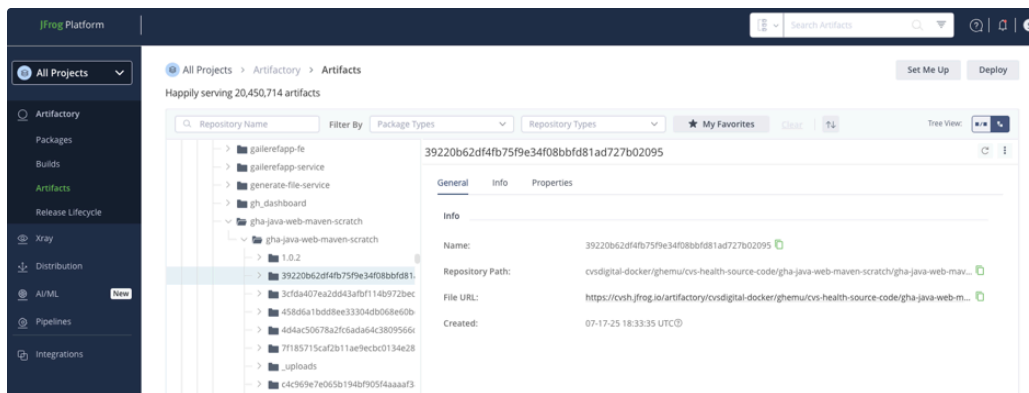
Steps to push the image to JFrog:

- Navigate to “Actions” tab and click “Run Docker Pipeline” workflow and select the “Run workflow” dropdown to trigger the CI process by clicking on “Run workflow” button.

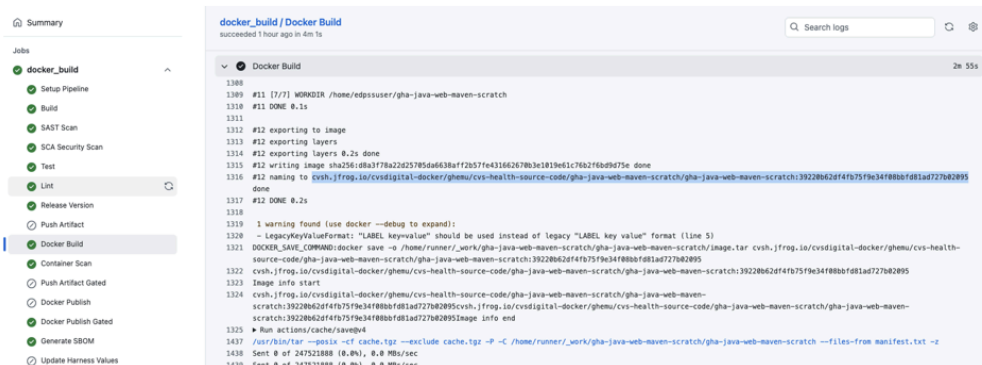


- This process will go through all the stages in CI to push your application image to jfrog registry.

In this example, it will push it to this location `cvsh.jfrog.io/cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch`



The version of the image can be found from the gha workflow logs under docker build section as well.



Refer Snyk CI setup for GHA

<https://aetnao365.sharepoint.com/sites/Application-Ecosystem-Security-and-Threat-Management/SitePages/CI-Setup.aspx> Connect your OneDrive account

CD

Recommended Folder Layout for CD workflow

```
1 name: App Deployment
2
3
4 on:
5   workflow_dispatch:
6   push:
7     branches: [gha-deploy]
8     paths: ['**/values/v1.yaml']
9
10 permissions:
11   id-token: write
12   contents: write
13   pull-requests: write
14
15 jobs:
16   detect-changes:
17     runs-on:
18       group: cvs-linux-self-hosted
19     outputs:
20       CHANGED_ENVIRONMENTS: ${ steps.set-matrix.outputs.CHANGED_ENVIRONMENTS }
```

```

21   - uses: actions/checkout@v4
22     with:
23       fetch-depth: 0
24
25   - name: Detect changes
26     id: changes
27     uses: dorny/paths-filter@de90cc6fb38fc0963ad72b210f1f284cd68cea36
28     with:
29       base: ${GITHUB_REF_NAME}
30       filters: |
31         scratch-two-dev_v1: '**/deploy-configs/scratch-two-dev/values/v1.yaml'
32         scratch-two-pdev_v1: '**/deploy-configs/scratch-two-pdev/values/v1.yaml'
33         scratch-two-prod_v1: '**/deploy-configs/scratch-two-prod/values/v1.yaml'
34
35   - name: Build Environment Matrix
36     id: set-matrix
37     run: |
38       ENVS=()
39
40       if [ "${{ steps.changes.outputs.scratch-two-dev_v1 }}" = "true" ]; then
41         ENVS+=("${ENV_NAME}:\scratch-two-dev\", \"HELM_FILE_VERSION\": \"v1\")"
42       fi
43
44       if [ "${{ steps.changes.outputs.scratch-two-pdev_v1 }}" = "true" ]; then
45         ENVS+=("${ENV_NAME}:\scratch-two-pdev\", \"HELM_FILE_VERSION\": \"v1\")"
46       fi
47
48       if [ "${{ steps.changes.outputs.scratch-two-prod_v1 }}" = "true" ]; then
49         ENVS+=("${ENV_NAME}:\scratch-two-prod\", \"HELM_FILE_VERSION\": \"v1\")"
50       fi
51
52       if [ ${#ENVS[@]} -gt 0 ]; then
53         JSON=$(IFS=,; echo "${ENVS[*]}")
54         echo "CHANGED_ENVIRONMENTS=$JSON" >> $GITHUB_OUTPUT
55       else
56         echo "CHANGED_ENVIRONMENTS=[]" >> $GITHUB_OUTPUT
57       fi
58
59   deploy:
60     uses: cvs-health-source-code/gitops-cd-workflow/.github/workflows/helm-deploy.yaml@v1
61     needs: detect-changes
62     if: needs.detect-changes.outputs.CHANGED_ENVIRONMENTS != '[]'
63     strategy:
64       fail-fast: false
65       matrix:
66         environment: ${fromJson(needs.detect-changes.outputs.CHANGED_ENVIRONMENTS)}
67     secrets: inherit
68     with:
69       app-name: "gha-java-web-maven-scratch"
70       environment: ${matrix.environment.ENV_NAME}
71       helm-template-repo: ${vars.HELM_TEMPLATE_REPO}
72       helm-file-version: ${matrix.environment.HELM_FILE_VERSION}
73       helm-template-branch: main
74       canary-weight: "0"
75       lob: "account"
76       change-description: "EDP SS demo application change for production"
77       rfc-short-desc: "This is to deploy EDP SS demo application to production"
78       change-domain: "Enterprise"

```

DevEx Documentation: [CD Workflow](#)

Sample Projects

<https://github.com/cvs-health-source-code/gha-java-web-maven-scratch> Connect your Github

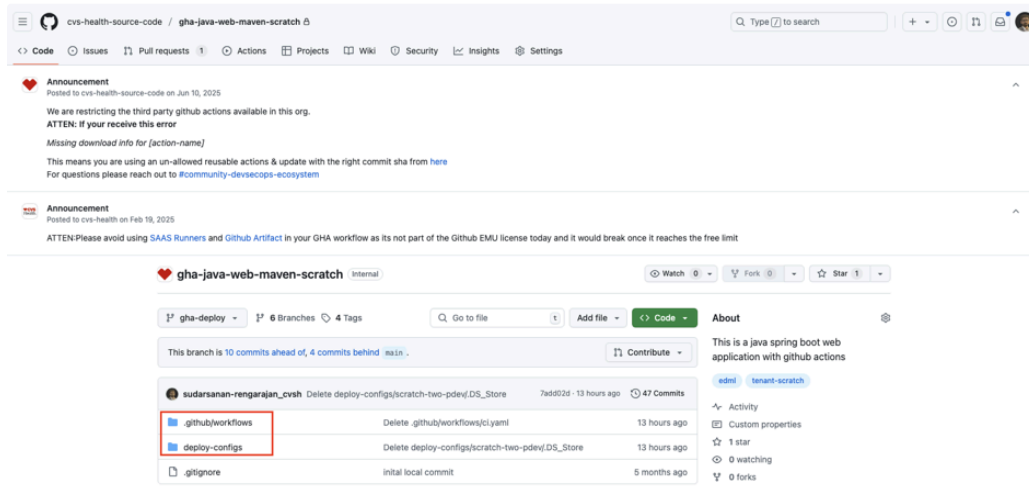
ub account

<https://github.com/cvs-health-source-code/edp-ss-python-flask-demo/tree/main> Connect y

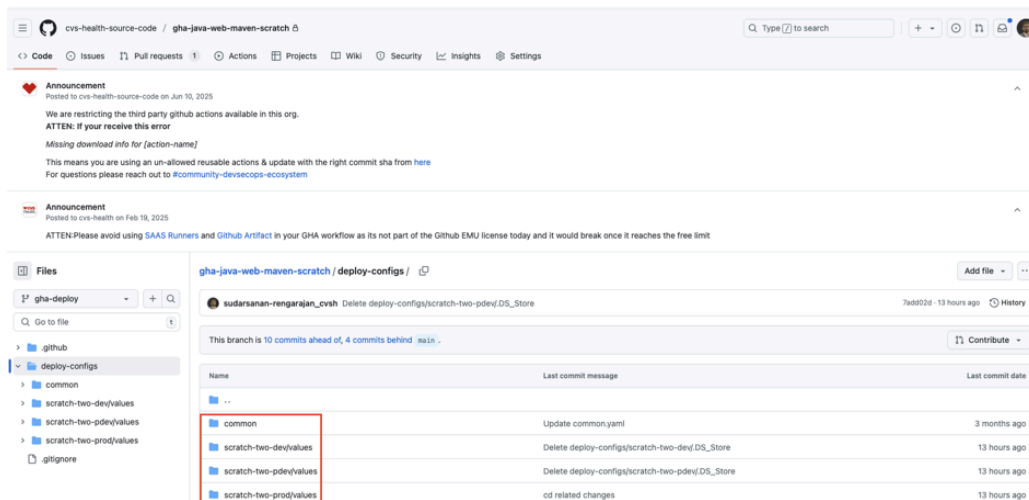
our Github account

Configuration

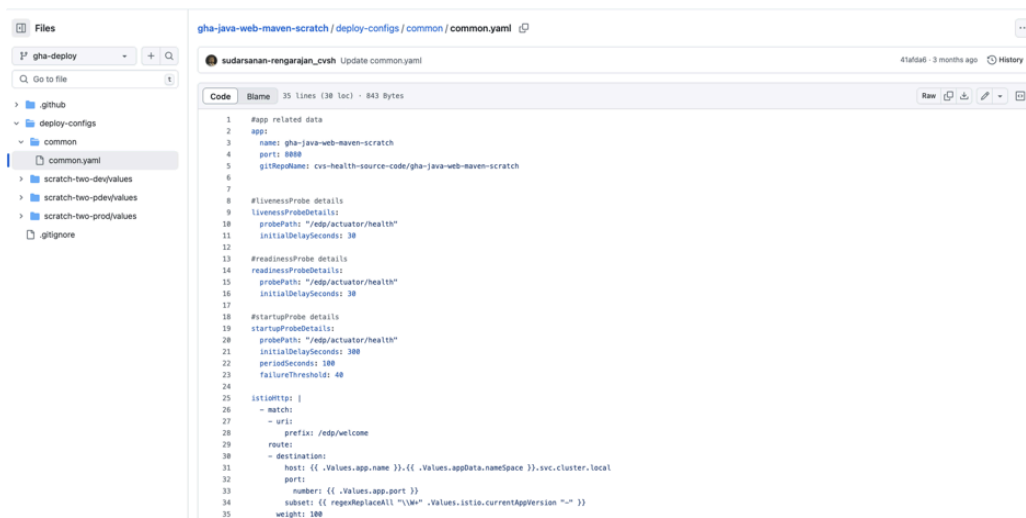
To hold the deployment configuration for Tenants, create a branch `gha-deploy` and create these two folders `./github/workflows` & `deploy-configs`.



`deploy-configs` directory generally has child directories `common` and `<namespace_{n}>/values`.



common : This directory holds the configuration spec values which are common across all the app deployments.



The screenshot shows a file explorer on the left with the following structure:

- gha-deploy
 - deploy-configs
 - common
 - common.yaml
 - scratch-two-dev/values
 - scratch-two-pdev/values
 - scratch-two-prod/values
 - .gitignore

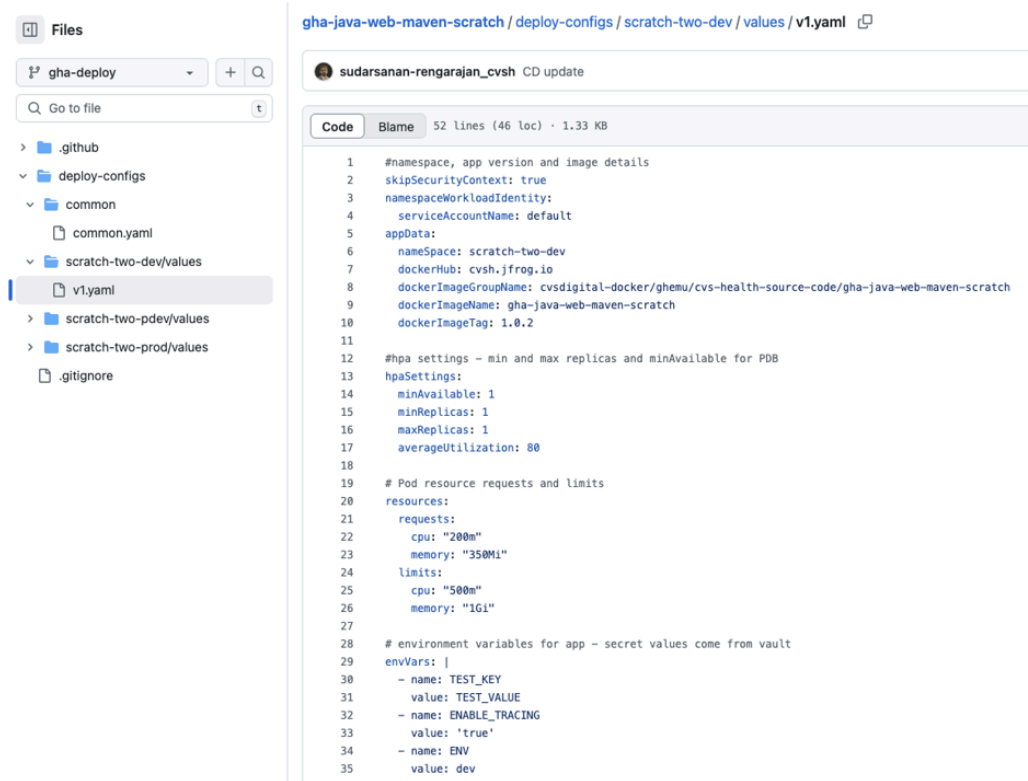
The main editor displays the content of `common.yaml`:

```
1 #app related data
2 app:
3   name: gha-java-web-maven-scratch
4   port: 8888
5   gitRepoName: cvs-health-source-code/gha-java-web-maven-scratch
6
7 #livenessProbe details
8 livenessProbeDetails:
9   probePath: "/edp/actuator/health"
10  initialDelaySeconds: 30
11
12 #readinessProbe details
13 readinessProbeDetails:
14   probePath: "/edp/actuator/health"
15   initialDelaySeconds: 30
16
17 #startupProbe details
18 startupProbeDetails:
19   probePath: "/edp/actuator/health"
20   initialDelaySeconds: 300
21   periodSeconds: 100
22   failureThreshold: 40
23
24
25 istioHttp: |
26   - match:
27     - uri:
28       prefix: /edp/welcome
29   route:
30     - destination:
31       host: {{ .Values.app.name }}-{{ .Values.appData.namespace }}.svc.cluster.local
32       port:
33         number: {{ .Values.app.port }}
34       subset: {{ replaceRepaceAll "\\.Values.istio.currentAppVersion ~" "" }}
35   weight: 100
```

namespace : This directory will hold the application specific configuration values (ex: v1, v2 etc.).

Here in below example **scratch-two-dev** is the namespace in this reference screenshot.

To initiate the deployment of your application, you need to update the **dockerImageTag** at **line 10** as in the below screenshot, with the latest Image tag, that got generated by your last CI workflow run.



The screenshot shows a file explorer on the left with the following structure:

- gha-deploy
 - deploy-configs
 - common
 - common.yaml
 - scratch-two-dev/values
 - v1.yaml
 - scratch-two-pdev/values
 - scratch-two-prod/values
 - .gitignore

The main editor displays the content of `v1.yaml`:

```
1 #namespace, app version and image details
2 skipSecurityContext: true
3 namespaceWorkloadIdentity:
4   serviceAccountName: default
5 appData:
6   namespace: scratch-two-dev
7   dockerHub: cvsh.jfrog.io
8   dockerImageGroupName: cvsdigital-docker/ghemu/cvs-health-source-code/gha-java-web-maven-scratch
9   dockerImageName: gha-java-web-maven-scratch
10  dockerImageTag: 1.0.2
11
12 #hpa settings - min and max replicas and minAvailable for PDB
13 hpaSettings:
14   minAvailable: 1
15   minReplicas: 1
16   maxReplicas: 1
17   averageUtilization: 80
18
19 # Pod resource requests and limits
20 resources:
21   requests:
22     cpu: "200m"
23     memory: "350Mi"
24   limits:
25     cpu: "500m"
26     memory: "1Gi"
27
28 # environment variables for app - secret values come from vault
29 envVars: |
30   - name: TEST_KEY
31     value: TEST_VALUE
32   - name: ENABLE_TRACING
33     value: 'true'
34   - name: ENV
35     value: dev
```

Naming Standards

To ensure consistency, observability, and ease of automation, all GKE resource names across tenants should follow these naming patterns:

- [Introduction](#)
 - [Manifest Repo](#)
 - [Namespace Configuration](#)
 - [Application Endpoints Configuration](#)
 - [One Time Setup for Every Repository](#)
 - [GitHub Repo Structure & Standards](#)
 - [Source Code Repo](#)
 - [CI](#)
 - [Recommended Folder Layout for CI workflow](#)
 - [DevEx Documentation: CI Workflow Step-By-Step Process](#)
 - [Sample Projects](#)
 - [Jfrog](#)
 - [CD](#)
 - [Recommended Folder Layout for CD workflow](#)
 - [DevEx Documentation: CD Workflow](#)
 - [Sample Projects](#)
 - [Configuration](#)
 - [Naming Standards](#)
 - [ArgoCD](#)
 - [References](#)

Resource Type	Naming Pattern	Example
Namespace	<lob>-<appAcronym>-<envname>	edml-edpss-dev edml-edpss-qa edml-edpss-pdev edml-edpss-pqa edml-edpss-prod

Note:

- Please be informed that you appcronym which is retrived from service now will be trimmed to 30 char.

ArgoCD

Every cluster will have a ArgoCD endpoint, which is of pattern `https://internal-argocd-prod.$CLUSTER_NAME.cvshealth.com` . where you can track the

applications deployed on the GKE cluster.

More documentation on ArgoCD: [📖 ArgoCD App of Apps](#)

References

Recording can be found [here](#).